

kqagent - How it guards bad q from being written

The authoring-time guard (the Maze lint engine wired into Claude Code hooks). This is a **separate system from Aegis**: it governs how the agent WRITES q, not what happens when q RUNS. The clean separation is stated in the limits below.

The core mechanism (the bit the demo hides)

Claude Code fires **PreToolUse** hooks before it executes any tool. A hook that exits non-zero (specifically **exit code 2**) makes Claude Code **refuse the tool call** and hand the hook's message back to the model. For a Write/Edit/MultiEdit on a .q file, 'refuse the tool call' means **the file is never written to disk**. That is the whole enforcement: it isn't the model choosing to behave - it's the harness declining to carry out the write, deterministically, and telling the model why so it can fix-and-retry.

So 'guards bad code from being written' is literal: the bad version never lands on disk; the model only gets to save a version the gate accepts.

What's actually wired (three hooks, three jobs - only two block)

- 1 **The Maze lint engine - the hard code block.** `checkKdbLint()` (the same function `tools/gate.js` wraps for the CLI/demo) runs on the proposed content of every Write/Edit. If the edit introduces a **new block-severity violation**, Maze's PreToolUse hook denies the write. This is the layer that stops =on-a-string or a sym-first partition query from ever being saved. 58 rule detectors (regex + AST) in `rules-kdb/`, defined in `catalog.json`.
- 2 `skill_gate_hook.py` - **the workflow block.** A second PreToolUse hook. It exits 2 to block a refine write unless `tools/search.py` was called recently (a tier-based minimum number of searches). It doesn't judge the code - it forces the draft → search → refine discipline so refinements are informed by the skill index, not by the model guessing.
- 3 `pretooluse_q_gate.py` - **advisory, does NOT block.** It runs the lint to predict which rules would fire, then injects those rule messages + the top-3 skill snippets into the model's context as `additionalContext`, so the model sees the fix before it writes. Its own docstring is explicit: 'The hook does NOT block.'

So: one hook hard-blocks bad code (Maze), one hard-blocks skipping the skill step, one feeds the fix. The first is the 'can't write bad q' guarantee; the others make the model fix it well.

The baseline / bootstrap detail (important and honest)

The gate is a 'don't make it worse' gate, not a 'this file is now clean' gate:

- The **first write to a fresh file is allowed** (the draft phase - drafts are ungated by design).
- On that first touch it captures all pre-existing violations as a **baseline**.
- Subsequent writes are blocked **only if they introduce a new block-severity violation beyond that baseline**. Pre-existing bad code is not retroactively blocked.

The honest limits (what the senior should hear)

- **Only inside a hook-enabled Claude Code session.** A human editing the file, CI, or any other tool isn't in the loop. It governs the agent's authoring, not the filesystem.
- **Static lint, not verification.** It catches what the 58 rules know. A mistake no rule encodes passes silently. It does not run the q.
- **Scoped to .q/.kl/.quke.** Writing q under another extension dodges the matcher - a known evasion surface mitigated only by convention (CLAUDE.md explicitly forbids it).
- **Syntactic, so gameable on intent.** The `date within (2000.01.01, 2099.12.31)` catch is the proof: the model satisfied the rule's form (date predicate first) while defeating its purpose. The gate checks structure, not selectivity or intent.
- **The skill-gate forces a search, not comprehension.** It can verify a `search.py` call happened; it can't verify the model read or applied it.
- **Authoring-time only.** It makes no runtime or security guarantee - nothing about who may run the query, on which rows, or whether injected input drives it. *That is Aegis's job; this is the clean separation between the two systems.*

One line: "kqagent stops the agent from SAVING bad q (a deterministic Claude Code hook that refuses the write); Aegis governs what happens when q RUNS (who, which rows, injected input). Authoring guard versus runtime gate - two systems, one clean boundary."